

Introducción a MongoDB

En esta práctica haremos una introducción al uso de la base de datos NoSQL de tipo documental MongoDB. Probaremos la funcionalidad que proporciona para la creación de documentos, la lectura, modificación y borrado de los mismos y el uso de operadores de agregación de datos para su procesamiento. El uso de la replicación y el sharding para configurar y usar un cluster con MongoDB lo veremos en la siguiente práctica.

En el caso de MongoDB, en lugar de crear una imagen de un contenedor usando las instrucciones de instalación para Ubuntu, tal y como hicimos con PostgreSQL, vamos a usar directamente una imagen proporcionada en el hub de docker. Para crear el contenedor ejecutamos el siguiente comando.

```
docker run --name xine-grei-mongodb -dti -p 27017:27017 --platform=linux/amd64 -v c:\temp:/home/alumnobd/host-temp mongo:latest
```

Por defecto, esta imagen mongo:latest ejecuta el proceso servidor de MongoDB, llamado mongod, usando los parámetros por defecto. La IP en la que escucha es la 27017, por lo que debemos mapear ese puerto si queremos conectar desde el host, por ejemplo para usar el cliente gráfico Compass, que podemos instalar en nuestra máquina. Puedes obtener este cliente en la siguiente URL para tu sistema operativo: <https://www.mongodb.com/try/download/compass>.

Para poder usar la shell de MongoDB lo más fácil es abrir una terminal en el contenedor y después ejecutar la shell *mongosh*.

```
docker exec -it xine-grei-mongodb bash  
mongosh
```

En la shell podemos especificar el nombre o IP de la máquina y el puerto de distintas formas.

```
mongosh "mongodb://mongodb0.example.com:28015"  
mongosh --host mongodb0.example.com:28015
```

?

```
mongosh --host mongodb0.example.com --port 28015
```

La conexión con un Replica Set o con un clúster vía *mongos* la veremos más adelante cuando tratemos la replicación y el sharding.

Los siguientes comandos básicos nos permiten ver la base de datos actual, cambiar la base de datos actual, ver las bases de datos del servidor o borrar la base de datos actual.

Ver la base de datos actual:	<code>db</code>
Usar otra base de datos:	<code>use <base de datos></code>
Listar las bases de datos del servidor:	<code>show dbs, show databases</code>
Listar las colecciones de una BD:	<code>show collections db.getCollectionNames() db.getCollectionInfos()</code>
Borrado de la base de datos actual:	<code>db.dropDatabase()</code>

Las bases de datos en general se crean al insertar datos en ellas por primera vez. Se puede cambiar a una base de datos inexistente y al crear los datos se crea la base de datos. La shell permite insertar comandos de varias líneas, usando {, (, [, hasta que todos estén cerrados no se termina el comando. Se puede usar el tabulador para completar expresiones, y las flechas del teclado para acceder al historial.

Vamos a ver ahora algunos ejemplos de operaciones usando la shell

Operaciones de gestión de datos (Inserción, Lectura, Modificación y Borrado)

Ahora veremos algunos ejemplos de operaciones de distintos tipos directamente sobre documentos que vamos a teclear en la shell. Más detalles en la página del manual.

Inserción de documentos

Las inserciones se hacen en una única colección. Las operaciones son atómicas a nivel de documento. MongoDB añade un campo `_id` en cada documento, si este no ha sido proporcionado por el usuario. Se puede especificar un compromiso de escritura concreto, pero esto lo veremos en una sección posterior.

```
use xine

db.empleados.insertOne(
  {nombre: "Miguel Angel Pintor",
    direccion: {calle: "Via del Corso",
                numero: "14",
                cp: "23452",
                localidad: "Roma"},
    paga_mensual: [23000, 23500, 23800, 23800, 23500, 22900, 23200, 46000, 23100, 23700, 23500, 46100]}
)
```

Esta función inserta un único documento, y el parámetro que se le pasa es un objeto JSON. Se crea automáticamente la colección empleados, y se inserta su primer documento. Para ver este documento podemos hacer una consulta.

```
db.empleados.find().pretty()
```

Al añadir la función “**pretty()**” el texto del resultado se muestra con el sangrado correcto para su mejor visualización. Puede observarse como se añade un nuevo campo `_id` con un identificador de objetos nuevo.

Si necesitamos insertar varios elementos podemos usar la función “**db.coleccion.insertMany()**”, a la que se pasa como parámetro un array de objetos JSON.

```
db.empleados.insertMany([
  {nombre: "Leonardo Ingenioso",
    direccion: {calle: "Leonardo Ingenioso",
      numero: "5",
      cp: "24987",
      localidad: "Roma"},
    paga_mensual: [12,24,18,12,143,56,34,34,64,456,34,756,34,14,23]
  },
  {nombre: "Donato di Niccolo di Betto Bardi",
    direccion: {calle: "Via dei Coronari",
      numero: "28",
      cp: "23452",
      localidad: "Roma"},
    paga_mensual: [23000, 23500, 23800, 23800, 23500, 22900, 23200, 46000, 23100, 23700, 23500, 46100]
  },
  {nombre: "Rafael Sanzio",
    direccion: {calle: "Via Margutta",
      numero: "34",
      cp: "23456",
      localidad: "Roma"},
    paga_mensual: [12,12,42,34,23,123,24,23,124,434,45,345]
  }
])
```

Recordamos que, en este caso, al atomicidad se mantiene a nivel de documento, por lo tanto, una operación de lectura podría leer el primer documento insertado antes de que el segundo se haya insertado.

Existen otras operaciones que permiten la inserción de documentos en MongoDB, así como métodos específicos para la importación y exportación, como veremos más abajo.

Consulta de documentos

Para poder realizar consultas vamos primero a insertar algunos documentos de ejemplo. Obtenemos los documentos de los datos de películas insertados en PostgreSQL, y generamos una llamada a la función `db.peliculas.insertMany()`, que guardaremos en un archivo de texto `peliculas.js` que almacenamos en la carpeta `public`.

```

select 'db.peliculas.insertMany('||
    cast(json_agg(json_build_object('titulo', p.titulo,
                                    'presupuesto', p.presupuesto,
                                    'emision', p.fecha_emision,
                                    'ingresos', p.ingresos,
                                    'generos', p.generos,
                                    'directores', (select json_agg(empleado->>'name')
                                                    from creditos c , json_array_elements(c.personal) as
personal(empleado)
                                                    where c.pelicula = p.id
                                                    and empleado->>'job' = 'Director'
                                                    ),
                                    'reparto', (select json_agg(json_build_object('nombre', ereparto->'name',
'personaje', ereparto->'character')
                                                    order by cast(ereparto->>'order' as
integer)
                                                    )
                                                    from creditos c, json_array_elements(c.reparto) as reparto(ereparto)
                                                    where p.id = c.pelicula
                                                    )
                                    )
    ) as text)
    ||')'
from peliculas p
where presupuesto > 250000000

```

Para cargar los datos en mongo, ejecutamos desde dentro la shell la siguiente función.

```
load("/home/alumnogreibd/public/peliculas.js")
```

Ahora deberíamos de tener 9 películas en la colección **peliculas**. Empezamos realizando una consulta con una condición e igualdad. Obtener los datos de la película titulada "John Carter".

```
db.peliculas.find({titulo: "John Carter"}).pretty()
```

Obtener el número de películas que tienen un presupuesto mayor que 275 millones.

```
db.peliculas.find({presupuesto:{$gt:275000000}}).count()
```

La lista de operadores que pueden usarse en las consultas se describe en el manual. Si se incluyen varias condiciones, se combinarán usando el operador lógico AND.

```
db.peliculas.find({presupuesto:{$gt:275000000},
                  ingresos:{$lt:1000000000}}).pretty()
```

Si necesitamos realizar un cálculo durante la consulta, o comparar dos campos del documento, entonces necesitamos utilizar el operador \$expr. Con la siguiente consulta obtenemos el número de películas posteriores a una determinada fecha (necesitamos obtener la fecha del string que tenemos almacenado).

```
db.peliculas.find({
  $expr: {
    $gt: [{
      $toDate: "$emision"
    }, ISODate("2012-01-01")]
  }
}).count()
```

Se pueden referenciar directamente campos de documentos anidados usando la notación "**campo1.campo2**", y también se puede consultar dentro de arrays y arrays de documentos anidados. Con esta consulta devolvemos las películas que tienen un género con nombre "Western".

```
db.peliculas.find({"generos.name": "Western"}).pretty()
```

Obtener las películas en las que "Johnyh Depp" ha interpretado el personaje "Tonto". Vemos dos formas alternativas de expresar la misma consulta.

```
db.peliculas.find({"reparto.nombre": "Johnny Depp", "reparto.personaje": "Tonto"}).pretty()
db.peliculas.find({reparto: {nombre: "Johnny Depp", personaje:"Tonto"}}).pretty()
```

El orden de los campos en la última de las consultas anteriores importa, y por lo tanto, esta consulta no devolvería nada.

```
db.peliculas.find({reparto: {personaje:"Tonto", nombre: "Johnny Depp"}}).pretty()
```

Para obtener los documentos en los que algún elemento del array cumple un conjunto de condiciones, sin tener en cuenta el orden, también se puede utilizar el operador "\$elemMatch"

```
db.peliculas.find({
  reparto: {
    $elemMatch: {
      personaje: "Tonto",
      nombre: "Johnny Depp"
    }
  }
}).pretty()
```

Se puede proyectar el resultado para obtener solo algunos de los campos. En la siguiente consulta nos quedamos solo con los campos título y presupuesto de las películas que tienen un presupuesto mayor de 275 millones.

```
db.peliculas.find({
  presupuesto: {
    $gt: 275000000
  }
}, {
  titulo: 1,
  presupuesto: 1
}).pretty()
```

El número 1 indica que se incluye el campo. También se pueden excluir campos, usando el número 0. No se puede mezclar la inclusión y la exclusión de campos, salvo para excluir el campo `_id` en una proyección de inclusión.

```
db.peliculas.find({
  presupuesto: {
    $gt: 275000000
  }
}, {
  titulo: 1,
  presupuesto: 1,
  _id:0
}).pretty()
```

```
db.peliculas.find({
  presupuesto: {
    $gt: 275000000
  }
}, {
  reparto: 0,
  generos: 0,
  _id:0
}).pretty()
```

Se puede proyectar también dentro de documentos anidados en arrays.

```
db.peliculas.find({
  presupuesto: {
    $gt: 275000000
  }
}, {
  titulo: 1,
  presupuesto: 1,
  reparto: {
    nombre: 1
  }
}).pretty()
```


Modificación de documentos

Para modificar un único documento se utiliza la función `db.collection.updateOne()`. Con el siguiente código modificamos los ingresos de la película "Avengers: Age of Ultron".

```
db.peliculas.updateOne(
  { titulo: "Avengers: Age of Ultron" },
  {
    $set: { ingresos: 1405403695 }
  }
)
```

Además del operador `$set` se pueden usar otros operadores para realizar la modificación. Por ejemplo, con el operador `$push` podemos añadir uno o varios elementos a un array.

```
db.peliculas.updateOne(
  { titulo: "Avengers: Age of Ultron" },
  {
    $push: { generos: {id:999, name: "Marvel Film"} }
  }
)
```

Además de la función anterior existen otras funciones para modificar el contenido de los documentos. Por ejemplo, `db.collection.updateMany()` permite modificar todos los documentos que cumplen la condición, y `db.collection.replaceOne()` reemplaza el primer documento que cumple la condición.

Se pueden utilizar pipelines de agregación para realizar modificaciones. Un pipeline está formado por etapas, donde cada etapa realiza una operación en los documentos antes de pasárselos a la etapa siguiente. Las etapas que pueden usarse en operaciones de modificación son seis. Veremos como se procesan datos con estos pipelines en una sección posterior. Ahora solo algunos ejemplos de como se utiliza esto para hacer modificaciones.

Modificamos el campo "emision" para transformarlo de al tipo de datos "date".

```
db.peliculas.updateMany({}, [{
  $set: {
    emission: {
      $dateFromString: {
        dateString: "$emision"
      }
    }
  }
}])
```

En el array de etapas hemos usado solo la etapa set. En el siguiente ejemplo usamos dos etapas para transformar el array de directores en un campo director, para las películas que tienen solo un director.

```
db.peliculas.updateMany({
  $expr: {
    $eq: [{
      $size: "$directores"
    }, 1]
  }
}, [{
  $set: {
    director: {
      $first: "$directores"
    }
  }, {
    $unset: "directores"
  }
}])
```

Borrado de documentos y de colecciones

Para borrar varios documentos se utiliza `db.coleccion.deleteMany()`, y para borrar un único documento de los que cumplen la condición usamos `db.coleccion.deleteOne()`. Por ejemplo, vamos a borrar primero una de las películas en las que "Johnny Depp" tuvo el papel de "Captain Jack Sparrow".

```
db.peliculas.deleteOne({
  reparto: {
    $elemMatch: {
      nombre: "Johnny Depp",
      personaje: "Captain Jack Sparrow"
    }
  }
})
```

Ahora borramos todas las películas del genero "Science Fiction".

```
db.peliculas.deleteMany({"generos.name": "Science Fiction"})
```

Terminamos borrando la colección completa.

```
db.peliculas.drop()
```

Si lo que necesitamos es borrar la base de datos.

```
db.dropDatabase()
```

Importación de documentos desde archivos JSON o CSV

La herramienta mongoimport nos permite cargar datos en formatos JSON y CSV de un archivo a una base de datos MongoDB.

Empezamos con un ejemplo en el que importamos documentos almacenados en formato JSON. En este caso, cada línea del archivo es un documento JSON. Para generar los datos vamos a usar una consulta SQL cuyo resultado vamos a exportar a formato csv, sin cabeceras, para que coincida con el formato JSON. En la exportación eliminamos las comillas.

```
psql -d xine -c "\copy
(select json_build_object('id', id,
                        'titulo', titulo,
                        'coleccion', coleccion,
                        'presupuesto', presupuesto,
                        'generos', generos,
                        'fecha_emision', fecha_emision)
from peliculas
limit 20)
to 'peliculas1.json'
with csv quote E'\r'"
```

Para importar los datos usamos la herramienta mongoimport con la siguiente sintaxis básica.

```
mongoimport --db=xine --collection peliculas1 --file=peliculas1.json
```

Si los datos JSON están almacenados como un array, entonces necesitamos utilizar la opción `--jsonArray`.

```
psql -d xine -c "\copy
(select json_agg(json_build_object('id', id,
                        'titulo', titulo,
                        'coleccion', coleccion,
                        'presupuesto', presupuesto,
                        'generos', generos,
                        'fecha_emision', fecha_emision))
from peliculas
where id < 50)
to 'peliculas2.json'
with csv quote E'\r'"
```

```
mongoimport --db=xine --collection peliculas2 --jsonArray --file=peliculas2.json
```

Para importar datos en formato CSV tenemos que usar la opción `--type=csv` y la opción `--headerline` para indicar que el archivo tiene una línea de cabecera.

```
psql -d xine -c "\copy
(select id, titulo, fecha_emision, presupuesto, ingresos
from peliculas
where id < 50)
to 'peliculas3.csv'
with csv header"
```

```
mongoimport --db=xine --collection peliculas3 --type=csv --headerline --file=peliculas3.csv
```

Preferencias y compromisos de lectura y escritura

Describimos ahora muy brevemente las preferencias de lectura y los compromisos de lectura y escritura que pueden usarse con MongoDB. Estos parámetros tienen impacto en el rendimiento de las lecturas y escrituras y también en la posible consistencia de la base de datos. Los compromisos de lectura y escritura se utilizan para obtener el compromiso apropiado entre consistencia y disponibilidad.

Preferencias de lectura

La [preferencia de lectura](#) se puede cambiar en los parámetros de conexión. Se puede cambiar también en la shell usando la siguiente función.

```
db.getMongo().setReadPref('nearest')
```

Las posibles preferencias son las siguientes:

primary

Preferencia por defecto. Las lecturas se hacen del nodo primario del replica set.

primaryPreferred

Intenta leer del primario, pero si este no está disponible, lee de algún secundario.

secondary

Las lecturas leen de nodos secundarios del replica set.

secondaryPreferred

Intenta leer del secundario, pero si ninguno está disponible entonces leer del primario.

nearest

Se lee de un nodo cualquiera (primario o secundario) en base a un cálculo de latencia previo.

Consistencia causal

Dentro de una sesión de usuario, con MongoDB se puede conseguir consistencia causal, es decir, operaciones futuras ven el efecto de operaciones pasadas. Ejemplo: Si hago un borrado con una condición y después una consulta con esa misma condición, la consulta no debería de devolver ningún resultado. Detalles sobre este tema se pueden encontrar en el manual, en la sección "[Read Isolation, Consistency, and Recency](#)".

Si las operaciones de lectura y escritura se ejecutan con el compromiso "Majority", entonces hay consistencia causal dentro de la misma sesión de usuario. En concreto, las garantías son las siguientes.

Read your writes

Cuando se escribe algo, una lectura posterior obtiene el valor actualizado

Monotonic reads

Una lectura no puede obtener un estado previo a una lectura anterior

Monotonic writes

Las escrituras se ejecutan en orden

Writes follow reads

Las escrituras que se han de realizar después de una lectura, se hacen en ese orden

Las operaciones de una sesión con consistencia causal no está aisladas de otras sesiones.

Compromiso de lectura (Read Concern)

El nivel de [compromiso de lectura](#) se especifica con una función adicional sobre los métodos `find`, `count`, `distinct`, etc.

```
db.collection.find().readConcern('majority')
```

Los niveles definidos son los siguientes

"local"

No verifica que se hayan escrito en la mayoría de los nodos. Es el nivel por defecto para lecturas del primario en sesiones de consistencia causal.

"available"

No verifica que se hayan escrito en la mayoría de los nodos. Valor por defecto para lecturas de secundarios fuera de sesiones de consistencia causal. Proporciona una latencia menor cuando hay sharding. La diferencia con "local" se da solo cuando hay sharding. En este caso, no se espera a tener consistencia en las particiones, por ejemplo, cuando se hacen migraciones.

"majority"

La lectura recupera datos que han sido comprometidos en la mayoría de los nodos del replica set. Los datos leídos son durables.

"linearizable"

La lectura obtiene los datos comprometidos antes de que se haya iniciado la operación. Puede tener que esperar por escrituras en curso. No se puede usar con transacciones. Solo se puede usar con lecturas del primario.

"snapshot"

Solo se puede usar con transacciones. Lee datos comprometidos por la mayoría.

Compromiso de escritura (Write Concern)

El nivel de [compromiso de escritura](#) se especifica con una estructura que tiene los siguientes campos.

w

w:1. Se necesita confirmación de escritura en el primario (o el nodo único si no hay replicación). Las escrituras en los secundarios se hacen después. **w:0.** No necesita confirmación de escritura. **w:mayor que 1.** Conformación del primario y varios secundarios (w:2. primario y un secundario). **w:majority.** Confirmación de la mayoría de nodos. **w:nombre específico.** Se necesita confirmación de un nodo concreto.

j

Confirmación de que se ha escrito en el on-disk journal (algo así como el registro histórico de MongoDB)

wtimeout

Solo cuando w es mayor que uno. Si se sobrepasa el límite de tiempo especificado, se genera un error.

El compromiso de lectura se puede especifica a nivel de transacción, si se usan transacciones, o a nivel de operación si no se usan transacciones. Se puede especificar un compromiso de escritura y lectura global, que será el usado cuando no se especifica en la operación o transacción. Este nivel global se especifica con un comando de administración.

```
db.adminCommand(  
  {  
    setDefaultRWConcern : 1,  
    defaultReadConcern: { <read concern> },  
    defaultWriteConcern: { <write concern> },  
    writeConcern: { <write concern> },  
    comment: <any>  
  }  
)
```

Agregación

Las operaciones de [agregación](#) en MongoDB procesan los documentos y obtiene colecciones de resultado. Existen tres formas de realizar agregaciones en MongoDB, los pipelines de agregación, las funciones map-reduce y los métodos de agregación de propósito específico, que incluyen la cuenta de documentos `db.collection.estimatedDocumentCount()` y `db.collection.count()` y la obtención de valores distintos de un campo `db.collection.distinct()`.

Para poder probar algunos ejemplos de agregación de distintos tipos, vamos primero a importar algunos datos de prueba de películas.


```
psql -d xine -c "\copy
(select json_build_object(
    'id', p.id,
    'titulo', p.titulo,
    'presupuesto', p.presupuesto,
    'generos', p.generos,
    'idioma', p.idioma_original,
    'titulo_original', p.titulo_original,
    'sinopsis', p.sinopsis,
    'fecha_emision', p.fecha_emision,
    'ingresos', p.ingresos,
    'duracion', p.duracion,
    'reparto', c.reparto,
    'personal', c.personal
)
from peliculas p, creditos c
where p.id = c.pelicula)
to 'peliculas.json'
with csv quote E'\r'"
```

```
mongoimport --db=xine --collection peliculas --file=peliculas.json
```

Ya tenemos las películas cargadas en la base de datos. Ahora podemos usar los agregados de propósito específico para obtener algunos datos preliminares.

```
db.peliculas.estimatedDocumentCount()
db.peliculas.count()
db.peliculas.distinct("duracion")
```

Pipelines de agregación

Un pipeline de agregación consta de una secuencia de etapas, cada una de ellas aplica una operación al resultado de la anterior. Las posibles etapas (operaciones) se describen en la parte de referencia del manual. También en la referencia se describen todos los operadores que se pueden usar dentro de las etapas.

Vamos a ver primero un ejemplo en el que se utilizan 4 etapas: la etapa **\$match** para filtrar y después la etapa **\$group** para agrupar y agregar, después la etapa **\$sort** para ordenar y finalmente la etapa **\$limit** para quedarse con los 10 primeros documentos.

```
db.peliculas.aggregate([  
  $match: {  
    presupuesto: {  
      $gt: 1000000  
    }  
  }, {  
    $group: {  
      _id: "$idioma",  
      peliculas: {  
        $sum: 1  
      },  
      ingresos_medios: {  
        $avg: "$ingresos"  
      }  
    }  
  },  
  {$sort: {peliculas:-1}},  
  {$limit: 10}  
])
```

Con la siguiente agregación procesamos los documentos para obtener la 10 directores que más ingresos han generado con sus películas.

```
db.peliculas.aggregate([{\n    $unwind: "$personal"\n}, {\n    $replaceRoot: {\n        newRoot: {\n            $mergeObjects: [{\n                ingresos: "$ingresos"\n            },\n                "$personal"\n            ]\n        }\n    }\n}, {\n    $match: {\n        job: "Director"\n    }\n}, {\n    $group: {\n        _id: "$name",\n        ingresos_totales: {\n            $sum: "$ingresos"\n        }\n    }\n}, {\n    $sort: {\n        ingresos_totales: -1\n    }\n}, {\n    $limit: 10\n}\n])
```

Primero la etapa `$unwind` desanida el array de personal de cada película. Cada documento del resultado tiene los datos de la película y los datos de un miembro del personal, en un documento anidado. La segunda etapa `$replaceRoot`, define como nuevo root de cada documento el documento anidado con los datos del miembro del personal, al que se le une, con el operador `$mergeObjects` los

ingresos de la película. En el resultado, ahora tenemos un documento para cada miembro del personal de cada película, con un campo que almacena los ingresos de la película. La siguiente etapa **\$match**, filtra los documentos para quedarse solo con los que tienen en el campo **job** el valor "Director". La etapa **\$group**, ya la hemos visto arriba, agrupa por el nombre de cada director y suma los ingresos. Finalmente, **\$sort** y **\$limit** se quedan con los 10 ingresos más altos.

Agregación con funciones map-reduce

Se recomienda el uso de pipelines de agregación en lugar de map-reduce por su mejor rendimiento, en general. Solo vamos a ver un ejemplo muy simple ya que no asumiremos conocimientos previos de JavaScript.

```
db.peliculas.mapReduce(  
  function(){  
    for (let i=0;i<this.generos.length;i++){  
      emit(this.generos[i].name, 1);  
    }  
  },  
  function(key, values) {return Array.sum(values)},  
  {query: {idioma:"es"},  
    out:"generos"  
  }  
)
```

La primera función JavaScript (map) genera pares (clave,valor). La segunda función (reduce) agrega los valores para cada clave. Con la opción **query** se filtran los documentos antes de pasárselos a las funciones. La opción **out** proporciona el nombre de la colección de salida.

Como puede verse, con map-reduce, se puede utilizar cualquier función JavaScript para realizar la agregación.

Última modificación: luns, 9 de febreiro de 2026, 19:26